EXPERIMENT REPORT

EXPERIMENT TITLE

# Case Study on the limitations of low-code platforms for a complex enterprise use case

241401117


SUBMITTED BY

| | |
|---|---|
| Name | **Tharika R** |
| Register Number | **241401117** |

# 1. INTRODUCTION

Low-code development platforms enable users to develop applications with minimal manual coding through visual interfaces, drag-and-drop components, predefined templates, and workflow automation tools. These platforms reduce development time and make application development accessible to users with limited programming knowledge.

However, developing complex enterprise applications using low-code platforms can introduce several limitations. These include restricted customization, integration difficulties, scalability constraints, security requirements, vendor dependency, and challenges in implementing advanced business logic.

This experiment focuses on analyzing the limitations of low-code platforms using a complex loan processing system as a case study. Various diagrams are used to understand the system architecture, identify platform limitations, analyze application complexity, compare alternative development approaches, and examine integration challenges.

# 2. AIM

To analyze the limitations of low-code platforms in developing complex applications using a loan processing system and to identify suitable alternative development approaches.

# 3. OBJECTIVES

1. To understand the proposed enterprise architecture of a loan processing application.
2. To identify the major limitations of low-code development platforms.
3. To analyze the relationship between application complexity and low-code platform suitability.
4. To compare low-code development with hybrid and custom development approaches.
5. To examine the challenges involved in integrating external systems with a low-code application.

24140117

# 4. SYSTEM OVERVIEW – PROPOSED ENTERPRISE ARCHITECTURE

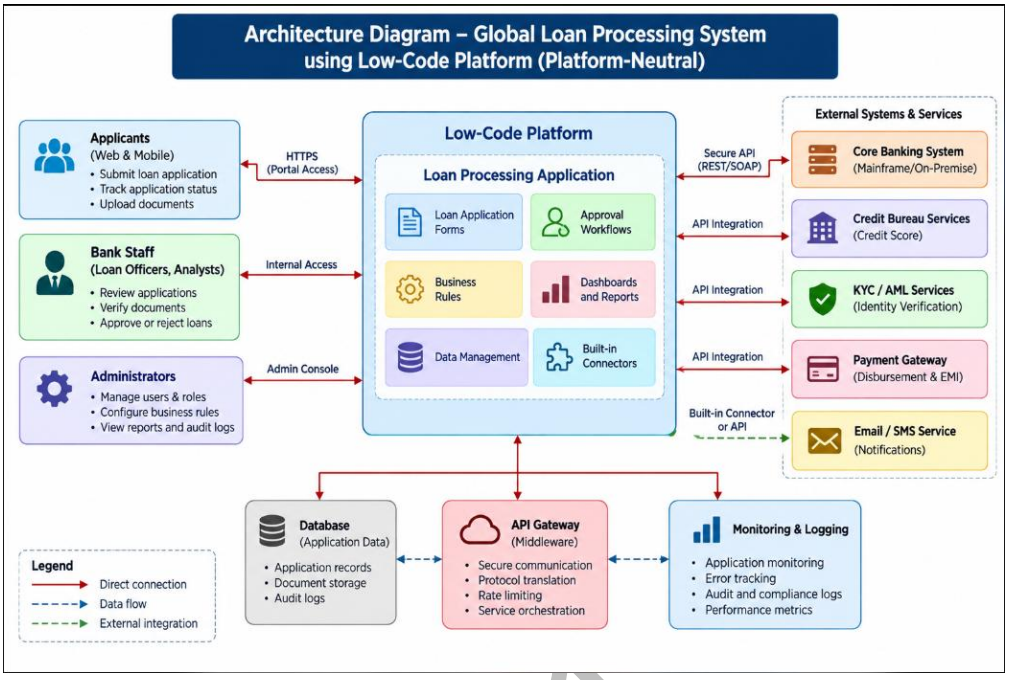



**Figure 1:** *Proposed Enterprise Architecture*

## Description

The proposed enterprise architecture illustrates the major components required to develop and operate a complex loan processing application using a low-code platform. The architecture includes applicants, bank staff, administrators, the low-code platform, external systems, a database, an API gateway, and monitoring services.

Applicants interact with the system to submit loan applications, upload supporting documents, and track application status. Bank staff review submitted applications, verify documents, and perform approval-related activities. Administrators manage user access, configure business rules, and monitor application operations.

The low-code platform provides essential features such as loan application forms, approval workflows, business rules, dashboards, reports, and data management. External systems, including core banking services, credit bureaus, KYC/AML services, payment gateways, and email/SMS services, can be connected through APIs or available integrations.

The database stores application information, document details, and audit records. An API gateway or middleware layer can support communication between the platform and external services, while monitoring and logging help track errors and system performance.

This architecture demonstrates that low-code platforms can manage standard application functions. However, complex business logic, advanced integrations, and demanding security or scalability requirements may require custom services alongside the low-code platform.

24140

### Component Analysis

| Component | Description |
|---|---|
| Applicants | Submit applications and supporting documents. |
| Bank Staff | Review applications and process loan requests. |
| Administrators | Manage users, permissions, and business rules. |
| Low-Code Platform | Provides forms, workflows, dashboards, and data management. |
| External Systems | Support banking, credit checks, identity verification, and payments. |
| Database | Stores application records and audit information. |
| API Gateway / Middleware | Coordinates communication between different systems. |
| Monitoring and Logging | Helps track failures, performance, and operational activities. |

## 5. LIMITATION ANALYSIS

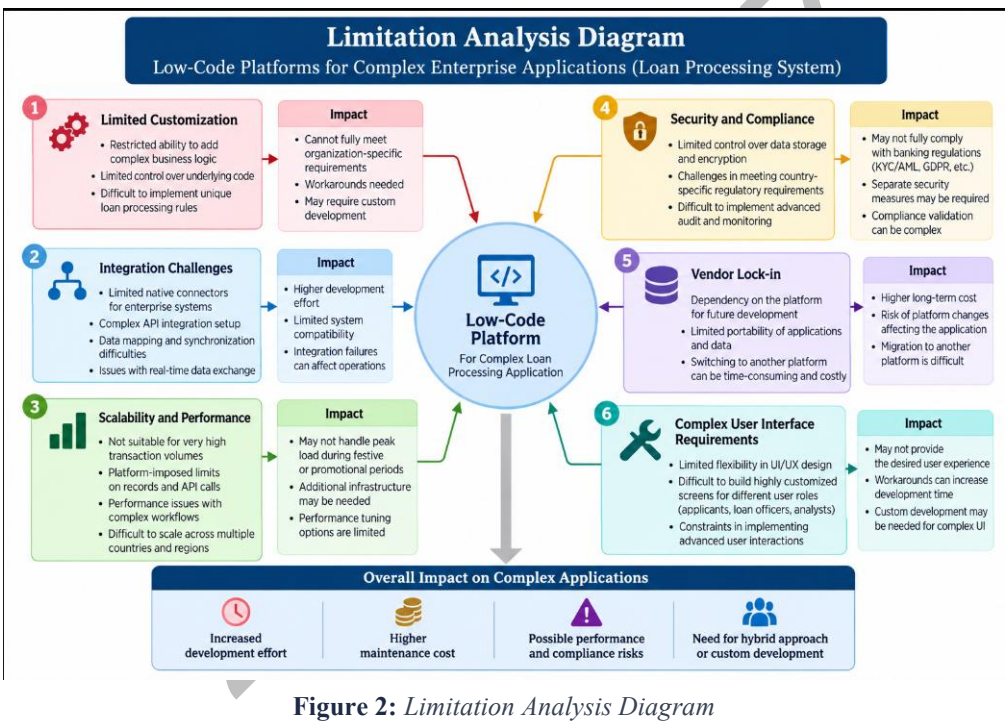



**Figure 2:** *Limitation Analysis Diagram*

### Description

The limitation analysis diagram identifies the major challenges that may arise when low-code platforms are used to develop complex enterprise applications. It presents six important areas: limited customization, integration challenges, scalability and performance, security and compliance, vendor lock-in, and complex user-interface requirements.

**1. Limited Customization:** Low-code platforms provide predefined components and development features. Implementing highly specialized business rules or unique application functionality may require workarounds or additional programming.

**2. Integration Challenges:** Complex applications often communicate with several external systems. Differences in API formats, authentication methods, and data structures can increase integration effort.

**3. Scalability and Performance:** Applications with high transaction volumes or demanding response-time requirements may encounter platform-specific limitations. Performance must be evaluated through suitable testing.

**4. Security and Compliance:** Enterprise applications may require strict access control, data protection, audit trails, and compliance procedures. These requirements must be supported by the platform and its configuration.

**5. Vendor Lock-In:** Applications developed using platform-specific components may be difficult to migrate to another platform. Migration can require rebuilding workflows, integrations, and application logic.

**6. Complex User Interfaces:** Advanced user interactions and highly customized screens may be difficult to implement using standard low-code components.

These limitations can increase development effort, maintenance costs, and implementation risks. Their actual impact depends on the selected platform and the application's requirements.

## Limitation Analysis Table

| Limitation | Possible Impact | Suggested Solution |
|---|---|---|
| Limited customization | Complex business rules may be difficult to implement. | Use custom services for specialized logic. |
| Integration challenges | Communication and data synchronization errors may occur. | Use APIs, middleware, and integration testing. |
| Scalability and performance | High workloads may affect response time. | Conduct load testing and review platform limits. |
| Security and compliance | Additional controls may be necessary. | Implement access control, encryption, and audit logging. |
| Vendor lock-in | Migration may become costly. | Document dependencies and evaluate portability. |
| Complex user interfaces | Standard components may not meet all requirements. | Introduce custom components where necessary. |

# 6. COMPLEXITY ANALYSIS – COMPLEXITY VS SUITABILITY GRAPH

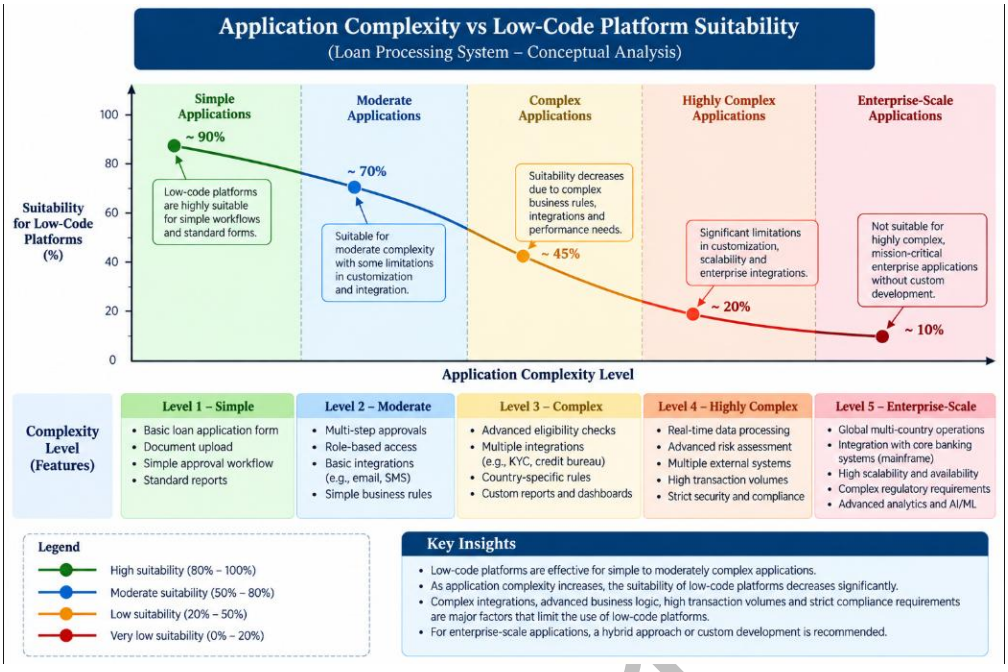



**Figure 3:** *Complexity vs Suitability Graph*

## Description

The graph illustrates the conceptual relationship between application complexity and the suitability of low-code platforms. It divides applications into five levels: simple, moderate, complex, highly complex, and enterprise-scale.

At Level 1, simple applications involving basic forms, document uploads, standard reports, and simple approval workflows are generally well suited to low-code development.

At Level 2, applications may require multi-step approvals, role-based access, basic integrations, and additional business rules. These requirements can often be implemented using low-code features, provided the platform supports them.

At Level 3, applications may include advanced eligibility checks, multiple integrations, and customized reports. Such applications require careful evaluation of platform capabilities and may benefit from a hybrid approach.

At Levels 4 and 5, requirements may involve real-time processing, advanced risk assessment, high transaction volumes, extensive security controls, and large-scale enterprise integrations. These requirements may exceed the capabilities of some low-code platforms.

The graph communicates the general idea that platform suitability may decrease as application complexity increases. The percentages displayed in the graph are illustrative conceptual values and are not based on actual experimental measurements.

241401

### Complexity Analysis Table

| Level | Complexity | Example Requirements | Possible Approach |
|---|---|---|---|
| Level 1 | Simple | Basic forms and standard workflows | Low-code |
| Level 2 | Moderate | Role-based access and multi-step approvals | Low-code with testing |
| Level 3 | Complex | Advanced business rules and multiple integrations | Hybrid approach |
| Level 4 | Highly Complex | Real-time processing and advanced risk assessment | Hybrid or custom components |
| Level 5 | Enterprise-Scale | Extensive integrations, high availability, and strict compliance | Evaluate hybrid or custom development |

## 7. ALTERNATIVE SOLUTIONS – DECISION FLOWCHART

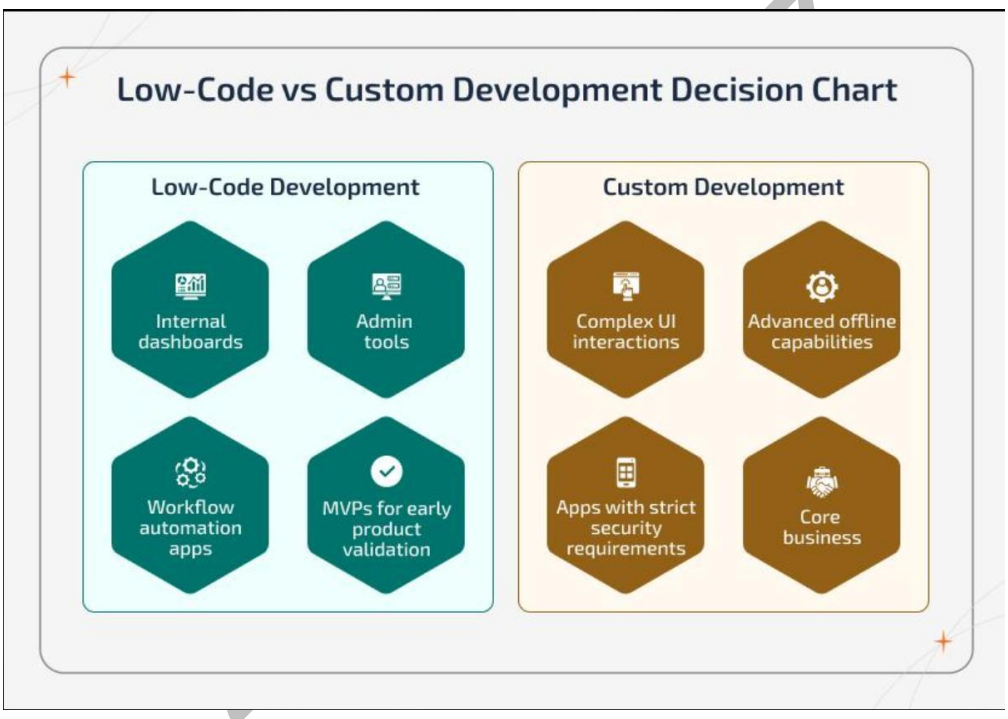



**Figure 4:** *Low-Code vs Custom Development Decision Chart*

### Description

The decision chart compares low-code development with custom development and identifies situations in which each approach may be suitable.

Low-code development is useful for applications such as internal dashboards, administrative tools, workflow automation, and minimum viable products. These applications can often be developed quickly using predefined components and visual development tools.

Custom development may be more appropriate when an application requires complex user interactions, advanced offline capabilities, strict security controls, or specialized business functionality that cannot be implemented effectively using the selected low-code platform.

A hybrid approach combines both methods. Standard forms, dashboards, and approval workflows can be developed using low-code tools, while complex business logic, specialized integrations, and performance-critical functions can be implemented using custom code or external services.

The selection of a development approach should consider functional requirements, complexity, security, integration needs, development time, maintenance costs, and scalability.

### Comparison of Development Approaches

| Development Approach | Advantages | Limitations |
|---|---|---|
| Low-Code Development | Faster development and reduced manual coding | Platform-dependent customization and functionality |
| Hybrid Development | Combines rapid development with custom functionality | Requires careful integration between components |
| Custom Development | Greater control over architecture and implementation | Usually requires more development time and technical expertise |

Based on the comparison, a hybrid approach may be suitable for complex loan processing when standard functions can be implemented using low-code tools but specialized requirements need additional programming.

## 8. INTEGRATION CHALLENGES – INTEGRATION MAP

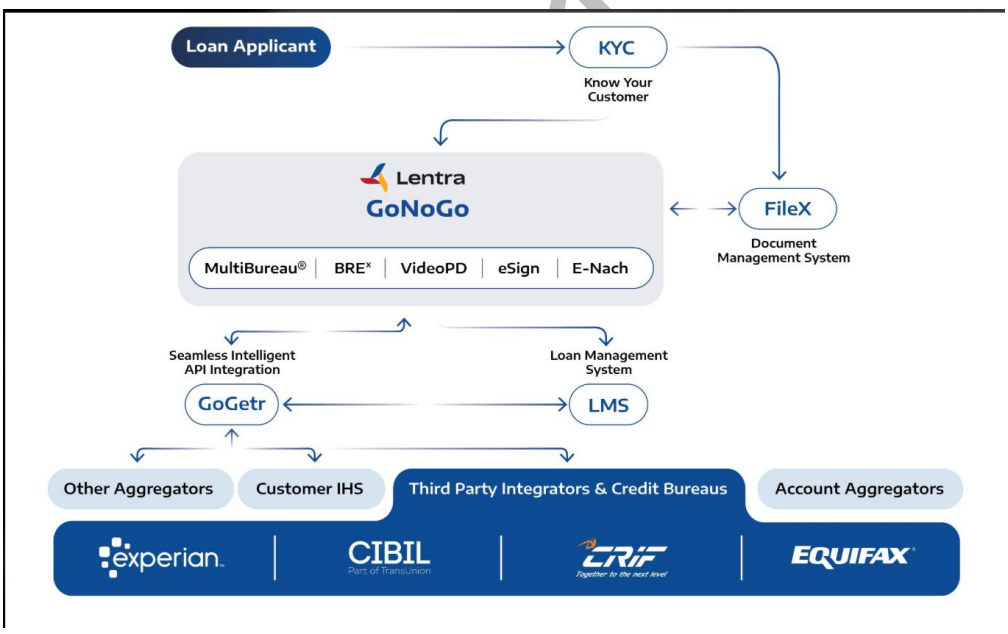



**Figure 5:** *Integration Map*

### Description

The integration map illustrates how a loan processing application communicates with external systems and services. These systems may include core banking, credit bureaus, KYC/AML verification, document storage, payment gateways, email/SMS services, and analytics or regulatory reporting systems.

Core banking systems provide banking-related information and support loan-related transactions. Credit bureaus supply credit information for evaluating loan applications. KYC/AML services support identity

verification and relevant compliance checks. Document management services store and retrieve supporting documents, while payment gateways handle payment-related operations.

Email and SMS services communicate application updates and notifications to applicants. Analytics and reporting systems can help administrators monitor application status and operational performance.

Integrating these systems can be challenging because each service may use different APIs, data formats, authentication mechanisms, and response procedures. Network failures or unavailable services can interrupt workflows or cause inconsistent application statuses.

These challenges can be reduced through secure API communication, input validation, appropriate error handling, retries, logging, access control, and end-to-end integration testing. Where built-in connectors are insufficient, middleware or custom integration services may be required.

### Integration Analysis Table

| Integration Component | Purpose | Major Challenge |
|---|---|---|
| Core Banking System | Provides banking information and supports loan transactions. | Data consistency and transaction reliability |
| Credit Bureau | Supplies credit history and credit-related information. | API availability and data accuracy |
| KYC/AML Services | Supports identity verification and compliance checks. | Sensitive data protection and compliance |
| Document Management | Stores and retrieves application documents. | Secure access and document handling |
| Payment Gateway | Processes payment-related transactions. | Failed transactions and reconciliation |
| Email/SMS Service | Sends application updates and notifications. | Delivery failures and retry handling |
| Analytics and Reporting | Supports monitoring and decision-making. | Data mapping and reporting accuracy |

## 9. OVERALL RESULT

The limitations of low-code platforms were analyzed using a complex loan processing system as a case study. The enterprise architecture identified the main application components and their interactions. The limitation analysis highlighted customization, integration, scalability, security, vendor dependency, and user-interface challenges.

The complexity graph demonstrated conceptually that the suitability of low-code platforms depends on the complexity of application requirements. The decision chart compared low-code, hybrid, and custom development, while the integration map illustrated the challenges involved in connecting external banking and support services.

The analysis indicates that low-code platforms can be effective for standard forms, workflows, and reporting. However, complex enterprise applications require careful assessment of platform capabilities, integration requirements, security, and performance.

# 10. CONCLUSION

This experiment examined the limitations of low-code platforms in developing complex applications through the example of a loan processing system. The analysis showed that low-code development can reduce development effort for standard application functions, but it may introduce challenges when extensive customization, multiple external integrations, high scalability, or strict security requirements are involved.

A hybrid development approach can provide a practical balance by using low-code tools for standard functionality and custom programming for specialized requirements. The final development approach should be selected after evaluating application complexity, platform capabilities, integration needs, security requirements, performance expectations, and long-term maintenance.

Thus, understanding the limitations of low-code platforms helps developers choose an appropriate architecture and build applications that are reliable, secure, maintainable, and suitable for enterprise requirements.

2414011117